

CULTURALLY ROBUST DRIVER DROWSINESS DETECTION

Students:

Aleen Mohammad Al-qwaifel	431201500
Joud Ibrahim Al-saweed	431201495
Raihanah Yousef Al-salom	421202111
Lana Suliman Al-deaiji	431201365
Leen Abdulaziz Al-jasser	431201485

Supervisor:

Dr. Aicha ben Makhoul

*A project report submitted to Qassim University in partial fulfillment of
the requirements for the degree of B.Sc. in Computer Science*

Qassim - Saudi Arabia

1447 (2025/2026)

CERTIFICATE

It is certified that this project report has been prepared and written under my direct supervision and guidance. This project report is approved for submission for its evaluation.

Dr. Aicha ben makhlouf

DEDICATION

This project is dedicated to the College of Computer and the Computer Science Department at Qassim University, in appreciation of the academic guidance, facilities, and supervision that supported the development of this work. We also extend this dedication to our families, whose constant encouragement and support have accompanied us throughout our studies.

Aleen Mohammad Al-qwaifel

Joud Ibrahim Al-saweed

Raihanah Yousef Al-salom

Lana Suliman Al-deaiji

Leen Abdulaziz Al-jasser

ACKNOWLEDGEMENT

This project has constituted a significant academic and personal undertaking, strengthened our perseverance and contributed to our development as computer science students.

We would like to express our sincere gratitude to Dr. Aicha ben Makhlouf, our project supervisor, for her continuous guidance, valuable feedback, and encouragement throughout the development of this project. Her support and insightful comments have been essential in shaping the direction and quality of this work.

Our appreciation is also extended to the Computer Science Department and the College of Computer at Qassim University for providing the academic environment, resources, and facilities that enabled us to complete this project.

Finally, we would like to thank our families and colleagues for their patience, motivation, and support during our years of study and during the preparation of this report.

Aleen Mohammad Al-qwaifel

Joud Ibrahim Al-saweed

Raihanah Yousef Al-salom

Lana Suliman Al-deaiji

Leen Abdulaziz Al-jasser

CONTENTS

List of Tables	vi
List of Figures	vii
Abstract	viii
Chapter One	1
Introduction	1
1.1 Introduction.....	1
1.2 Project Scope.....	1
1.3 Aim and Objectives.....	2
1.4 Motivation.....	2
1.5 Project Plan and Schedule.....	3
1.6 Outline of the Report.....	4
Chapter Two	6
Literature Review.....	6
2.1 Introduction.....	6
2.2 Background	6
2.3 Existing Related Systems.....	8
2.4 Contribution	11
2.5 Summary	12
Chapter Three.....	13
Problem Analysis	13
3.1 Introduction.....	13
3.2 Problem Specification	13
3.3 System Analysis.....	15
3.4 Implementation and Evaluation Plan	18
Chapter Four	20
System Design	20
4.1 Introduction.....	20
4.2 System Design Specification.....	20
4.3 Design Architecture	23
4.4 Pseudocode/Algorithms/Scenario	28
Chapter Five	30
5. System Implementation.....	30
5.1 Introduction.....	30
5.2 Implementation and Evaluation Plan	30

5.2.1 Data Preparation	30
5.2.2 Model Development	30
5.2.3 System Integration.....	31
5.2.4 Testing and Evaluation	31
5.3 Technical Tools and Programming Languages	31
5.4 Code Structure.....	32
5.5 Discussion	34
5.5.1 Model Training Performance.....	34
Chapter Six.....	36
6. System Testing and Result Discussion	36
6.1 Introduction.....	36
6.2 Testing Description and Test Cases	36
6.2.1 Overview of Testing Procedure.....	36
6.2.2 Tested Approaches	37
6.2.3 Test Cases Considered	38
6.3 Results and Discussions	38
6.3.1 Results Comparison and Discussion	38
6.3.2 Final Model Training.....	40
6.3.3 Cultural Robustness	40
Chapter Seven.....	42
7. Conclusion and Future Work.....	42
7.1 Conclusion	42
7.2 Future Work	42
7.3 FYP Closing Remarks.....	43
References	45

LIST OF TABLES

Table 2: Results Comparison	7
Table 2: Results Comparison	39
Table 3: YOLO11n Results Comparison	40

LIST OF FIGURES

Figure 1.1: Gantt Chart of the Project Plan.....	3
Figure 4.1: System flow chart	23
Figure 4.3: System Interaction (Sequence Diagram)	25
Figure 4.4: System Class Diagram	25

ABSTRACT

Drowsy driving is widely recognized as a major cause of road accidents, and this issue becomes particularly critical in situations where cultural clothing partially obscures the driver's face. Most existing driver-monitoring systems rely on fully visible facial features or specialized hardware such as EEG sensors and infrared cameras, which limits their practicality, affordability, and suitability for local driving conditions. This project presents the design, implementation, and evaluation of an AI-based driver-monitoring system that detects drowsiness in real time using computer vision on a standard webcam-enabled device. The proposed system adopts a fine-tuned YOLO11n deep learning model for real-time eye-state classification, enabling more robust operation when the lower part of the face is covered by local attire such as the niqab or shemagh. The system implements two graduated alert levels based on consecutive closed-frame streaks and the PERCLOS metric, with audio alerts generated procedurally using NumPy and Pygame. All processing is performed entirely on-device to preserve privacy and ensure low-latency feedback without dependency on external servers. The system was trained on a hybrid dataset of 5,000 images combining publicly available samples with locally collected culturally representative images, achieving a final Precision of 0.9661, Recall of 0.9752, and mAP@0.5 of 0.9877 after 100 training epochs. Comparative evaluation of three YOLO variants confirmed YOLO11n as the most suitable architecture for the proposed system. The report covers the literature review, problem analysis, system design, implementation, testing, and result discussion, and outlines future directions including mobile-optimized or embedded deployment, larger dataset expansion, model optimization, and possible integration of additional driver-behavior indicators such as head-pose estimation.

CHAPTER ONE

INTRODUCTION

1.1 Introduction

Drowsiness and distraction are among the main reasons behind car accidents worldwide. According to the American Automobile Association Foundation for Traffic Safety, drowsy driving contributes to an estimated 328,000 crashes annually, including more than 100,000 injuries [1]. However, the actual number is believed to be significantly higher. These statistics highlight the seriousness of the problem and the urgent need for effective solutions. Many existing solutions like EEG or ECG sensors [2], [3], in-vehicle devices, or even self-reporting methods are not ideal because they are expensive, uncomfortable, and often inaccurate. Recently, computer vision and machine learning have offered new ways to handle this issue. Object detection systems such as YOLO (You Only Look Once) can track eye movements, head position, and the use of handheld devices to spot early signs of fatigue. Yet, most current systems require full facial visibility, especially the mouth area, which makes them unreliable in some cultures. In Saudi Arabia and other Gulf countries, local attire such as the niqab, hijab, shemagh, and masks often covers parts of the face. This cultural aspect makes it difficult for existing models to perform accurately. Our project was built with this challenge in mind. It introduces a YOLO-based model to detect driver drowsiness even when part of the face is covered. The system is designed as a lightweight, mobile-optimized solution that processes video locally on the host device to protect privacy. When drowsiness is detected, the driver receives audible alerts. By respecting local traditions and using the strengths of computer vision, our work aims to make driving safer and smarter across the GCC (Gulf Cooperation Council) countries.

1.2 Project Scope

- **Data Collection and Training:** Compiling a dataset of drivers dressed in local attire (niqab, hijab, masks, and shemagh) and an existing dataset of bare faces for generalization purposes so the system can operate in cultural environments.

- **Computer Vision and Machine Learning:** Computer vision and deep learning approaches to automatically detect significant indicators of drowsiness (i.e., eye closure) in cases of partial visibility of the face.
- **Lightweight and Accessible Design:** The system is designed with mobile compatible, lightweight architecture to be affordable, reachable, and convenient without requiring high-value hardware components. The design supports future deployment on mobile or embedded devices.
- **Alert Systems:** Effective real-time alert systems (audio) activate to alert drivers of detected drowsiness.
- **Validation and Testing:** Practical testing of the system to know the detection capability, robustness, and usability.

1.3 Aim and Objectives

The project aims to develop an AI-based driver-monitoring system that can detect drowsiness in real-time using computer vision, even when the driver's face is partially covered by typical local attire (niqab, hijab, shemagh) and masks. By providing timely warnings at the moment risky behavior is detected, the overall goal is to improve road safety and help prevent accidents in culturally specific driving environments. To achieve this aim, the project will focus on the following key objectives:

1. Design and develop a deep learning model (YOLO) for drowsiness detection based on eye closure.
2. Collect and preprocess a diverse dataset by including typical local attire (niqab, hijab, shemagh) and masks to ensure robust and accessible system performance.
3. Design and optimize a lightweight system that monitors and provides real-time drowsiness feedback, structured to support future mobile deployment.
4. Provide alert systems such as audible alerts to notify the driver as soon as dangerous behavior is noticed.
5. Assess the overall system performance and use common metrics (e.g., precision, recall, mAP@0.5) to determine reliability.

1.4 Motivation

Drowsiness is widely documented as a major factor contributing to road traffic accidents worldwide, and the issue is equally critical within our region. Although various countries

have adopted camera-based driver-monitoring systems to mitigate this risk, such technologies demonstrate limited effectiveness locally. This limitation arises primarily from typical local attire that obscures essential facial features and prevents conventional computer-vision systems from accurately assessing the driver’s state of alertness. These constraints underscore the necessity of developing a solution that is both technologically feasible and culturally adaptable. Within the scope of this graduation project, the aim is to propose and implement an alternative detection mechanism that does not depend on full facial visibility. The system is designed as a lightweight, localized solution optimized for standard camera-equipped devices, utilizing intelligent vision-based algorithms to identify early indicators of drowsiness and reduced alertness levels. By providing a culturally compatible and accessible approach, this project seeks to contribute to reducing fatigue-related crashes and enhancing overall road safety within the region.

1.5 Project Plan and Schedule

Our project plan was structured to guide the development of the system through its complete lifecycle. We began by defining the project scope and conducting a focused literature review to identify current research gaps. This guided the problem analysis and system design phases. Following the architectural design, the project moved into the implementation phase, which involved custom dataset preparation, annotation, and the training of the YOLO11n deep learning model. Subsequently, the system integration phase connected the trained model with the real-time detection logic and audio alert mechanisms. Finally, rigorous testing and evaluation were conducted to validate the system's cultural robustness, culminating in the final documentation. The overall timeline of these tasks is illustrated in Figure 1.1.

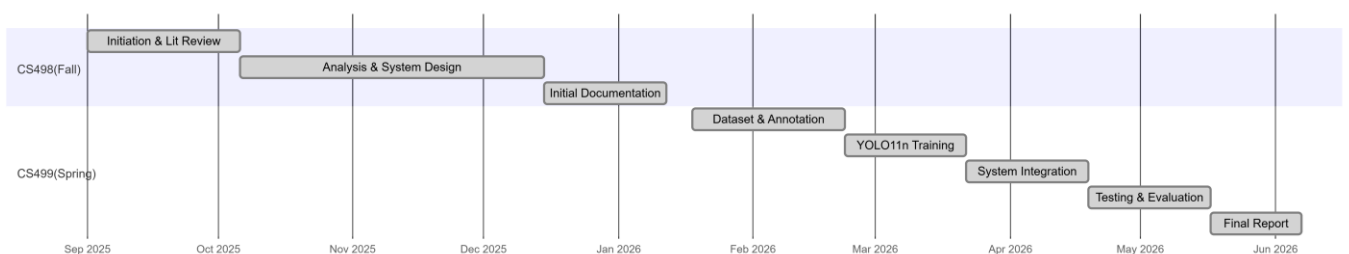


Figure 1.1: Gantt Chart of the Project Plan

1.6 Outline of the Report

This report is organized into seven chapters that collectively describe the research, analysis, design, and implementation of the proposed Driver Drowsiness Detection System. The remainder of this report is structured as follows:

- Chapter Two: Literature Review establishes the theoretical foundation of the project by covering key concepts in computer vision and driver-monitoring systems. It provides a critical analysis of existing related works, highlighting their limitations regarding cultural adaptability and testing conditions, and defines the research gap that this project aims to address.
- Chapter Three: Problem Analysis delves into the specific challenge of detecting drowsiness when facial features are partially obscured by local attire. It details the process of requirement collection and analysis, specifying the essential functional and non-functional requirements, and outlines the planned approach for implementation and evaluation.
- Chapter Four: System Design presents the architectural design of the solution. It describes the system's core components including input capture, preprocessing, and the YOLO detection model, and illustrates the logical flow and algorithms used to ensure real-time performance and on-device privacy preservation.
- Chapter Five: System Implementation presents the practical realization of the designed system. It details the technical tools and programming languages used, describes the code structure across its functional modules, and discusses the model training performance results, validating the core technical approach proposed in the preceding chapters.
- Chapter Six: System Testing and Result Discussion presents the evaluation of the system. It describes the testing procedures, the approaches compared during development, and the test cases considered. It then discusses the results, including model selection, final training outcomes, alert behavior, and cultural robustness under real-world conditions involving local attire.
- Chapter Seven: Conclusion and Future Work summarizes the key findings and contributions of the project, reflects on the extent to which the initial objectives were achieved, and outlines directions for future enhancement, including mobile or embedded deployment, larger dataset collection, model optimization, and possible integration of additional driver-behavior indicators.

Finally, the report concludes with the References used throughout the study.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the relevant literature and situates our project within previous studies and technological developments. It examines key related works, systems, and methods, assessing their strengths and weaknesses to define the broader knowledge framework surrounding the project. Through this analysis, we identify research gaps and justify the contribution of our work, thereby providing a solid foundation for the design and implementation presented in later chapters.

2.2 Background

This section examines the theoretical and technical foundations critical to the project, with emphasis on key topics.

Computer Vision

The term Computer Vision refers to a field of artificial intelligence (AI) that enables machines to interpret and understand visual information from the world by processing images and videos [4]. Through techniques such as machine learning and deep learning, computer vision systems can extract meaningful features, recognize patterns, and make decisions based on visual data. Applications of computer vision include facial recognition, driver-assistance systems, medical image analysis, and robotics, reflecting its transformative role in modern technology and automation [5].

Drowsiness Detection

The term Drowsiness Detection refers to the automated process of identifying reduced alertness and fatigue levels in individuals, particularly in contexts where sustained attention is critical, such as driving or operating machinery [6]. This process relies on monitoring physiological and behavioral cues associated with fatigue, including prolonged eyelid closure, altered blink rate, reduced gaze stability, frequent yawning, and

head-nodding movements, which collectively signal a decline in cognitive responsiveness [7].

Driver-Monitoring System

The term Driver-Monitoring System (DMS) refers to an advanced in-vehicle safety technology designed to assess driver alertness, attention, and behavior in real-time to prevent accidents caused by drowsiness or reduced alertness. These systems typically use computer vision and artificial intelligence techniques to track facial landmarks, eye movements, head position, and other physiological indicators to evaluate the driver's state and issue warnings when signs of drowsiness are detected [8].

Convolutional Neural Networks

The term Convolutional Neural Network (CNN) refers to a class of deep learning models specifically designed to process and analyze visual data by automatically learning hierarchical feature representations from images [9]. CNNs use convolutional layers to extract spatial patterns such as edges, textures, and shapes through learnable filters, enabling the model to capture both low level and high-level visual features efficiently. This architecture significantly reduces the need for manual feature engineering and improves performance in tasks involving complex visual structures. Due to their ability to generalize across diverse image variations, CNNs have become essential in modern computer vision applications, including facial recognition, driver-monitoring systems, medical imaging, and object detection [10]. Their robustness, scalability, and capability to learn discriminative features make them a foundational component in many state-of-the-art vision-based safety and automation systems.

You Only Look Once

The term You Only Look Once (YOLO) refers to a family of real-time, single-stage object detection models that formulate detection as a single regression problem, directly predicting bounding boxes and class probabilities from an input image in one forward pass through a neural network [11]. Unlike two-stage detectors, YOLO performs localization and classification simultaneously, which significantly improves inference speed while maintaining competitive detection accuracy, making it suitable for time-critical applications [11]. Subsequent YOLO versions introduced architectural and

training improvements to enhance detection performance, especially for smaller objects and varied scales, while preserving real-time capability [12]. Due to this balance between speed and accuracy, YOLO is widely used in computer vision systems that require fast detection (e.g., identifying faces, eyes in driver-monitoring scenarios) [11], [12].

2.3 Existing Related Systems

Table1 : Comparison of State of Art Methods

Title	Detection Features Used	Method/Model Used	Input Type/Data Source	Device/Platform Used	Reference
Enhanced Driver Drowsiness Detection Using Deep Learning	Eyes blink frequency and closure, Mouth yawning behavior, Facial expressions, and Head movement (temporal changes and nodding patterns).	CNNs (InceptionV3, MobileNet, DenseNet, AlexNet)	NTHU-DDD, YawDD video datasets	Video camera + PC/GPU	[13]
Real-Time Driver Drowsiness Detection Using CNN, MediaPipe, and ML Classifiers	Facial landmarks from MediaPipe	CNNs (ResNet50, VGG16), ML classifiers	Facial landmarks from MediaPipe	Webcam + PC/GPU	[14]
A Comparative Study Using CNN, GAN, and Hardware Models	blink freq, eyelid-closure duration, yawning/mouth opening, facial landmarks (MTCNN), GAN augmentation.	CNN + GAN vs Arduino sensor	Video frames + synthetic GAN images	Arduino + camera + PC	[15]
IoT-Based Driver Drowsiness Detection Using CNN	blinking + eyelid closure + yawning + head tilt; HR/SpO ₂ /skin temp; steering + CO ₂ .	YOLOv5 + CNN + IoT physiological/environmental data	Kaggle images + IoT sensor suite	IoT sensors + camera	[16]
YOLOv5 and LSTM-Based Driver Drowsiness Detection	eye-closure duration, blink freq, yawning (over time), head posture, external hazards.	YOLOv5 + LSTM + emergency IoT protocol	RLDD, NTHU-DDD, hazard datasets	Onboard camera + vehicle system	[17]
Investigative Comparison of DL Models for Drowsiness Detection	eye closure/blink, yawning/head tilt, facial gestures (static frames).	CNN, VGG16/19, AlexNet, ResNet-9	15,000 facial images	PC/GPU	[18]
Real-Time Driver Drowsiness Detection Using Vision Transformer	Binary Eye State Classification (Open/Closed)	Vision Transformer (ViT)	Eye image dataset (84,900 images)	Python, OpenCV, ViT	[19]
Title	Detection Features Used	Method/Model Used	Input Type/Data Source	Device/Platform Used	Reference

Hybrid CNN-LSTM Model with Facial Feature and Behavioral Analysis	Eye Aspect Ratio, Mouth Aspect Ratio, Head Pose / nodding movement	Hybrid CNN + LSTM, MediaPipe	Facial images (frames), Driver face dataset (UTARLDD) + real-time camera feed	Python, TensorFlow, MediaPipe, OpenCV	[20]
Driver Drowsiness Detection Using YOLO	Face/eye and yawning detection	YOLO for real-time detection	Real-time camera input	Embedded camera + computing device (e.g., PC / Raspberry Pi)	[21]
ViT vs. CNN vs. YOLO for Drowsiness Detection	Eyes state, facial expressions, micro-movement	ViT, CNN, YOLOv3	RGB images 244x244, NTHU-DDD2 dataset (66,521 samples)	Real-time camera based vision system, GPU-supported ViT deployment	[22]
YOLOv8-Powered Driver-monitoring	Using phone, eating, smoking, adjusting radio, drowsiness	YOLOv8 (CSP-Darknet53, PANet)	AUC + dashboard images (30,000)	NVIDIA RTX 3090 GPU, PyTorch-based Ultralytics YOLOv8 framework, ADAS (Advanced Driver Assistance Systems)	[23]
Eye State Classification with CNN	Eye status, yawning, head movement	CNN for eye status, logistic regression, random forest, support vector machine	MRL, CEW, Kaggle	Adam optimizer, standard deep learning training pipeline	[24]
EfficientNet-KNN for Real-Time Driver Drowsiness Detection	Eyes, Head movement (eyelid closure, blinking, nodding)	EfficientNetB0 + KNN	ULg DROZY dataset (36 videos, 5-sec segments)	Automotive ECU (TinyEfficientNet)	[25]
Enhanced Deep Smart Intelligence-Based Driver Drowsiness Detection Using OSVFS with CNN	Eyes (eyelids, eye closure, blinking patterns)	ResNet-50 CNN + OSVFS	DRV-DS dataset (5,486 images)	Python + Jupyter Notebook	[26]

In recent years, a variety of methods have been proposed for driver drowsiness detection, most of which rely on deep learning architectures. Convolutional Neural Networks (CNNs) are commonly used to detect key facial features, particularly focusing on eye-state analysis, including open/closed eyes, blinking rates, and other fatigue indicators. Some systems integrate temporal models such as Long Short-Term Memory (LSTM) networks to capture behavioral patterns over time, while others explore hybrid architectures combining CNNs with object detection frameworks like YOLO to track the driver's face and head pose. Additionally, the introduction of Vision Transformers (ViT) for fine-grained eye-state classification has shown promising results, especially in large-scale datasets. Meanwhile, there are also attempts to incorporate Internet-of-Things (IoT) sensors, such as heart rate monitors and EEG, alongside computer vision models to improve detection by capturing physiological and environmental data in real-time.

However, despite their effectiveness in controlled environments, most existing systems face significant limitations when deployed in real-world driving scenarios. A primary challenge lies in the assumption that these models are trained on datasets with full facial visibility, which significantly limits their accuracy in regions with cultural attire (e.g., niqab, hijab, shemagh), where portions of the face are occluded. Furthermore, many systems are developed under ideal conditions with fixed lighting, clear visibility, and still drivers, which does not reflect the dynamic nature of actual road conditions, including variable lighting, head movements, and diverse driving environments. Additionally, many models rely on expensive and intrusive hardware like EEG or infrared cameras, making them impractical for everyday use.

The proposed method addresses these shortcomings by developing a YOLO-based model designed to detect drowsiness using only the eye region, focusing on cues that remain visible even when the face is partially covered. The system is implemented as a lightweight, on-device solution, to handle inference locally and efficiently, thereby reducing processing overhead and hardware costs. Additionally, this approach is specifically trained on a diverse dataset that includes drivers in local attire and real traffic conditions, making it more adaptable and reliable for real-world deployment, particularly in regions with cultural face coverings. By ensuring that the computational pipeline is executed locally rather than relying on external cloud servers, the method preserves user privacy and ensures low-latency, real-time feedback. This makes it a more practical, inclusive, and privacy-respecting solution compared to existing models, offering a significant improvement in both accessibility and real-world applicability.

Limitations

- Lack of Inclusivity

Most systems are not designed to work effectively for all users. Previous works relied on essentially uncovered face datasets, such as NTHU-DDD and RLDD. These models may achieve lower accuracy for drivers wearing typical local attire, as the facial features, like the mouth or jawline, are not visible. The focus on drivers with cultural face coverings means our project gives us an enhanced, more realistic drowsiness detection system that is inclusive.

- Controlled Environment Testing

Many existing drowsiness detection models are developed and evaluated under controlled laboratory conditions, where lighting is stable, the driver's face is clearly visible, and the head position remains relatively fixed. However, actual driving environments are more complex and unpredictable. Factors such as low or changing illumination, head movements, camera angle variations, and partial facial occlusion caused by local attire can reduce detection accuracy and limit the reliability of these models in practical deployment.

- Poor model generalization

Most existing datasets do not sufficiently represent drivers wearing traditional or cultural attire. Since many models are trained mainly on uncovered faces, their performance may decrease when applied to drivers whose faces are partially covered by the niqab, hijab, shemagh, or masks. This limits their reliability in local driving environments and highlights the need for a culturally adaptive model trained to handle partial facial occlusion.

- Dependence on Additional Hardware

Some studies improve detection performance by using EEG sensors or infrared cameras, which makes such systems expensive and impractical. Our project is software-based and uses a standard camera input to make the solution more accessible and affordable.

2.4 Contribution

As observed in previous research, most driver-monitoring systems depend on uncovered facial datasets and controlled environments, which limit their performance in real-world driving scenarios. In GCC countries, typical local attire further challenges the accuracy of these systems. Additionally, several studies rely on costly sensors or multi-camera setups, reducing practicality and accessibility. In contrast, our project introduces a culturally adaptive and lightweight driver-monitoring system designed specifically for local conditions. It employs a YOLO-based approach to detect drowsiness using only a standard camera-equipped device. The system also provides instant alerts through audio when drowsiness is detected, contributing to enhanced road safety. In summary, this work contributes an inclusive, affordable, and practical driver-monitoring approach that extends existing research by addressing cultural, environmental, and technical limitations present in GCC countries' driving context.

2.5 Summary

This chapter presented an overview of the theoretical background and related research that form the foundation of this project. It began by introducing key concepts such as computer vision, drowsiness detection, driver-monitoring systems, and the YOLO model, emphasizing their importance in improving road safety through artificial intelligence and real-time image analysis. The review then highlighted critical limitations in existing systems, including the lack of cultural representation, dependence on ideal testing conditions, limited dataset diversity, and reliance on costly sensors. These constraints reduce the effectiveness and accessibility of many driver-monitoring solutions, particularly in regions where traditional attire partially covers the face. To address these gaps, our project proposes a YOLO-based model that focuses on detecting driver drowsiness under realistic conditions using a standard camera. The system aims to be inclusive, lightweight, and affordable, while enhancing safety and applicability within GCC countries.

CHAPTER THREE

PROBLEM ANALYSIS

3.1 Introduction

This chapter focuses on understanding the problem our project aims to solve. It explains the key issues, why they matter, and where current solutions fall short. The goal is to define the problem clearly and identify the factors that contribute to its scope. Through this analysis, we outline the essential requirements needed for an effective solution and build the foundation for the system design presented in the following chapter.

3.2 Problem Specification

Drowsy driving is identified as one of the leading contributors to serious road accidents worldwide. Numerous studies emphasize that driver fatigue weakens attention, slows reaction time, and increases the likelihood of life-threatening incidents. Although existing driver-monitoring and warning systems have shown potential in reducing such risks, most of these technologies are designed and trained on datasets that do not reflect the characteristics of our local environment. In regions where local cultural attire such as the niqab, shemagh, and hijab is commonly worn, global monitoring models often struggle to accurately interpret facial cues and eye-closure patterns, which are key indicators used in detecting early signs of drowsiness. This mismatch between the data used to train current systems and the visual reality of local drivers reduces the reliability and effectiveness of these solutions. Therefore, the core problem lies in the absence of a driver-monitoring system that is both culturally aware and trained on real local data. Without such adaptation, detection accuracy remains limited, and early warning capabilities may fail at critical moments. Addressing this problem requires the development of an intelligent computer vision-based system capable of analyzing indicators of driver drowsiness in real time, while being robust to variations in cultural attire, lighting conditions, and diverse facial appearances. Such a solution would contribute to improved detection accuracy, enhanced safety interventions, and ultimately

a reduction in fatigue-related accidents within the local context. In addition to the proposed computer vision-based approach, several alternative solutions exist, such as steering wheel pressure sensors, lane departure warnings, or vehicle-based motion analysis. While these technologies offer some level of assistance, research consistently shows that they react only after the driver begins to drift or lose control, making them less effective in identifying early signs of fatigue. In contrast, a vision-driven solution provides a more proactive method by monitoring behavioral indicators such as eye closure, blinking frequency, and head movement before dangerous impairment occurs.

This solution offers several benefits:

- It improves safety by detecting early signs of drowsiness even in situations where global models struggle, especially under cultural or environmental constraints.
- It removes the need for additional specialized hardware such as IR sensors or EEG devices, making the system more practical and cost-efficient.
- It accommodates typical local attire without requiring drivers to adjust or remove it, ensuring comfort and inclusivity.
- It delivers higher accuracy by being trained on real, local representative data, resulting in a system that is more reliable and suitable for real-world deployment in the region.

Developing such a solution requires proficiency in computer vision, machine learning model development, image preprocessing techniques, real-time data handling, and understanding human-computer interaction principles in safety-critical environments. These skills are essential to design a robust system capable of delivering dependable performance in real driving scenarios.

3.3 System Analysis

3.3.1 Requirement Collection

The systematic process for gathering the necessary system specifications and constraints was primarily achieved through two key methods: **Literature Review Analysis** and **Goal-Driven Derivation**. The collected information served as the foundational input for the subsequent Requirement Analysis.

1. Literature Review Analysis

A comprehensive review of existing driver-monitoring systems (as detailed in Chapter Two) was conducted to identify common **technical approaches, performance metrics, and, most importantly, critical limitations**. Requirements were directly derived from the documented **gaps** in current research, specifically:

- **Cultural Limitations:** The requirement for **cultural adaptability** was established by identifying the need to address the challenges posed by typical local attire which obscures facial features.
- **Testing Limitations:** The requirement for **robustness** was established by observing that most models perform poorly outside of controlled, ideal lab environments.

2. Goal-Driven Derivation

The project's high-level **Aim and Objectives** were decomposed into specific, measurable needs that the final system must fulfill. This ensured that every system requirement was directly aligned with the project's core purpose:

- **Detection Needs:** Requirements regarding the detection features (e.g., eye closure) and the need for a **YOLO-based model** were derived from Objective 1.
- **Operational Needs:** The requirement for **real-time performance** and **lightweight, mobile-optimized design** was derived from Objective 3, ensuring the system is affordable and convenient.
- **Output Needs:** The requirement for an **instant alert system** (audio) was derived

from Objective 4, which focuses on timely driver warnings.

These two approaches ensured that the collected requirements were comprehensive, technologically informed, and specifically tailored to solve the unique cultural and environmental challenges faced in the GCC (Gulf Cooperation Council) countries.

3.3.2 Requirement Analysis

Requirement analysis verifies that the defined requirements are clear, feasible, and aligned with the project aim and objectives. The requirements are organized into functional and non-functional categories, and for each one the source in the requirement collection process is identified. Consistency, dependencies, and key trade-offs between the requirements are also examined.

3.3.2.1 Functional Requirements Analysis

F1. Driver state detection (eye closure)

The system must detect eye closure as an indicator of drowsiness. This requirement is central to achieving the project's safety objective and supports all subsequent processing and alerting functions, as prolonged eye closure is among the most reliable and widely validated cues for fatigue detection in driver-monitoring research.

F2. YOLO-based detection model

The system employs a YOLO-based model to perform the required detection tasks, leveraging its capability for fast and accurate identification of relevant visual cues, such as eye closure, in real time. This approach provides accurate detection with a processing speed suitable for on-device real-time inference.

F3. Real-time monitoring and feedback

The system must operate in real time by processing incoming frames and providing feedback without noticeable delay. This is essential so that drowsiness is detected early enough for corrective action. Real-time performance is particularly important because the system is designed as a lightweight local solution intended for resource-constrained deployment.

F4. Instant alert mechanism (audio)

When drowsiness is detected, the system triggers immediate alerts using sound signals.

Alerts are clearly noticeable inside the vehicle cabin and directly linked to the current driver state, encouraging drivers to correct their behavior.

3.3.2.2 Non-Functional Requirements Analysis

NF1. Cultural adaptability and operation with partial facial occlusion

The system must function reliably for drivers wearing local attire such as niqab, hijab, shemagh, and masks, where large parts of the face are obscured. This requirement ensures that the system is appropriate for the Saudi and Gulf context and addresses a major limitation of existing research. It reflects the cultural and environmental considerations of traditional clothing and the resulting need for models capable of detecting drowsiness under limited facial visibility.

NF2. Robustness in real-world driving conditions

The system must maintain acceptable performance in real-world driving environments, beyond controlled laboratory settings. This includes varying lighting conditions, head movements, and camera positions. This requirement addresses the performance challenges observed in real-world scenarios during the literature review and reflects the identified need for robustness given the testing limitations found in prior work.

NF3. Localized processing and privacy preservation

The system ensures driver privacy by conducting all video analysis and model inference locally within a decentralized computing environment, ensuring that no raw visual data is permanently stored or transmitted to external servers.

NF4. Compliance with resource constraints (performance, storage, optimization)

The system must operate in real-time, processing incoming video frames and providing feedback with minimal latency so that drowsiness is detected early enough for corrective action. In addition, the system must respect the limited processing power, memory, and storage available on the deployment hardware; therefore, the deployed model must be suitably optimized for efficient on-device inference.

NF5. Affordability and accessibility

The solution should be deployable on standard camera-equipped devices without requiring specialized or expensive hardware, making it affordable and accessible to a

wide range of drivers. This requirement follows from the project's aim of developing a practical and accessible solution and from the identified operational needs that link the lightweight design to affordability and convenience. The system therefore relies only on a standard built-in camera and does not require additional specialized hardware, which reduces cost and increases accessibility for drivers. The architecture is designed to be lightweight and optimized, laying the groundwork for future mobile deployment.

NF6. Usability and user acceptance

The system must be acceptable to users by being automated, non-intrusive, and capable of providing timely, understandable alerts. High usability is necessary for users to keep the system enabled during normal driving. This requirement reflects user expectations that drivers need an automated, non-intrusive, privacy-preserving monitoring tool that can immediately alert them when drowsiness is detected. The application is therefore designed to be easy to install, configure, and use; alerts and messages are clear and understandable, and the system requires minimal user effort to remain active during normal driving, supporting long-term adoption.

3.3.2.3 Consistency, dependencies and trade-offs.

The functional requirements F1-F4 and non-functional requirements NF1-NF6 are interrelated. Accurate driver state detection using the YOLO-based model depends on cultural adaptability and robustness in real driving conditions. Real-time monitoring must be achieved while respecting on-device processing and hardware resource limitations. Alert mechanisms and interaction design are closely linked to usability and user acceptance.

Several trade-offs appear in the analysis. Increasing model complexity can improve detection accuracy but may reduce execution speed and increase energy consumption. Very sensitive detection may reduce missed unsafe events but can increase false alerts, which negatively affects user trust. These relationships guide the design choices and optimization strategies in the following chapter.

3.4 Implementation and Evaluation Plan

The implementation and evaluation plan outlines the steps required to develop the system and verify its performance during Phase 2. The plan follows a structured approach to

ensure that each component of the system is built, tested, and validated according to the defined requirements.

- **Data Preparation:** Collecting and preprocessing video samples that represent local driving conditions and cultural apparel variations.
- **Model Development:** Training and fine-tuning the YOLO-based model for detecting eye states.
- **System Integration:** Connecting the model with a real-time camera input and implementing the alert mechanism.
- **Testing and Evaluation:** Measuring detection performance using accuracy-based metrics and testing the system in simulated and real conditions.

Technical Tools and Programming Languages

Programming Language: Python selected as the primary programming language due to its extensive ecosystem in computer vision, machine learning, and deep learning research.

Deep Learning Models: A YOLO-based model is employed to detect eye states for drowsiness detection. Multiple variants are currently under evaluation to identify the most suitable configuration for real-time performance and computational efficiency.

Dataset Management: Public datasets such as NTHU-DDD and RLDD will be used as baselines. In addition, a custom dataset will be constructed to include drivers wearing local attire, addressing the limitations of existing datasets.

Development and Experimentation Environment: Google Colab / Kaggle Notebooks for GPU-accelerated training and prototyping.

Version Control and Documentation: GitHub for version control, collaboration, and code management, MS Word for maintaining project documentation and academic report writing.

CHAPTER FOUR

SYSTEM DESIGN

4.1 Introduction

This chapter presents the overall design and internal structure of the proposed Driver Drowsiness System. It explains how the system's components were organized, how they interact, and the approach used to process data in real-time on a standard camera-equipped device. The design focuses on practical deployment in GCC countries, where cultural attire may partially cover the face, and therefore emphasize reliable detection using the eye region.

4.2 System Design Specification

The proposed Driver Drowsiness Detection System is structured as a real-time pipeline that includes Input Capture, Frame Processing, YOLO-Based Detection, Alert Mechanism, and On-Device Processing & Privacy. Each component performs a specific function and forwards its output to the next stage, enabling real-time detection of driver drowsiness based on eye-state monitoring. The design emphasizes efficient on-device computation and robustness to typical local attire that may partially obscure facial features. The following subsections describe each component and its role in overall architecture.

Input Capture

The input capture module acquires live video of the driver using a front-facing camera. The device is positioned to provide a clear view of the driver's face. A continuous stream of frames (typically 15-30 frames per second) is provided for analysis. Given the cultural context in GCC countries, the system accounts for common attire that may obscure the lower face while often leaving the eye region visible. Accordingly, the capture configuration is designed to reliably acquire eye area ensuring that downstream modules receive sufficient visual information even when parts of the face are covered.

Preprocessing Module

After capture, the frame processing module prepares each frame for detection. Each captured frame is passed directly to the fine-tuned YOLO11 model, which performs inference on the live video stream. Instead of manually cropping the eye region or preparing separate inputs for different models, the YOLO model identifies the relevant eye-state class from the frame and returns the detected label with its confidence score. A confidence threshold is applied to filter weak detections and improve reliability.

The module also supports real-time visualization by drawing bounding boxes around detected regions and displaying the predicted class label, such as open or closed, with the confidence value. The processed output is then passed to the detection module, where the eye-state results are evaluated over time using consecutive-frame tracking and PERCLOS-based decision logic.

Detection Module

The detection module is the system's core, consisting of a fine-tuned YOLO11 deep learning model used for real-time eye-state detection. The model analyzes the live stream and classifies the detected eye region as either open or closed. This setup enables real-time monitoring of drowsiness through eye-closure detection.

YOLO Eye State Detection: The fine-tuned YOLO11 model processes each captured video frame and detects the eye state based on the trained classes: open and closed. The model uses a confidence threshold to filter weak detections and only accepts predictions that meet the minimum confidence value. For each valid detection, the system draws a bounding box around the detected region and displays the predicted label with its confidence score. This allows the system to visually indicate whether the driver's eyes are currently open or closed during live monitoring.

To reduce false alarms, drowsiness is not triggered from a single frame. Instead, the system tracks eye-closure behavior across consecutive frames and within a sliding window. If closed eyes are detected for a predefined number of consecutive frames, the system flags a drowsiness warning. In addition, the system calculates PERCLOS, which represents the proportion of frames in the sliding window where the eyes are detected as closed. If the PERCLOS value exceeds the defined threshold, the system activates the alert state.

The module also includes a critical alert condition. When the PERCLOS value reaches 100%, meaning all frames in the sliding window indicate closed eyes, the system enters a critical drowsiness state and triggers a stronger alarm. This layered detection logic helps distinguish between brief eye closures, sustained drowsiness, and severe risk conditions. Overall, the detection module provides practical real-time drowsiness detection by combining YOLO-based eye-state classification, consecutive-frame tracking, and PERCLOS-based decision logic.

Alert Mechanism

The system uses audio feedback and visual warning indicators to ensure that warnings are noticeable in real-time. When the driver's eyes remain closed for a predefined number of consecutive frames, or when the PERCLOS value exceeds the defined threshold, the system plays an audible warning. If the PERCLOS value reaches a critical level, the system triggers a stronger alarm to indicate a higher-risk drowsiness state.

The alert remains active as long as the detected risk condition continues. When the driver returns to a safe state, such as when the eyes are detected as open again and the drowsiness indicators fall below the threshold, the alarm stops and the system returns to normal monitoring. The primary objective of this mechanism is immediate safety, prompting the driver to regain alertness before the drowsy state becomes dangerous.

On-Device Processing and Privacy Handling

A core design specification of this project is its reliance on edge-aware methodologies. By integrating the YOLO11n model in an architecture explicitly designed to comply with mobile and edge resource constraints, the system handles frame capture, deep learning inference, and threshold decision-making locally within a real-time execution loop, eliminating any cloud computing dependency. This design reduces latency and allows the system to respond quickly when signs of drowsiness are detected.

From a privacy perspective, local processing helps limit the exposure of sensitive visual data because the video frames are processed directly during runtime. The frames are used for immediate inference, and the system only relies on derived indicators such as the detected eye state, the number of consecutive closed-eye frames, and the PERCLOS value for decision logic. This approach is important in contexts where facial privacy and cultural appropriateness are major concerns, especially when the system may capture partial facial regions during driver monitoring.

To support efficient real-time deployment, the system uses a compact detection pipeline based on a fine-tuned YOLO11 model with a confidence threshold to filter weak detections. The system tracks closed-eye frames over time using a sliding PERCLOS window and updates the alert status continuously. In addition, the interface displays bounding boxes, eye-state labels, FPS, closed-frame streaks, PERCLOS percentage, and warning overlays to provide real-time feedback. Overall, the proposed pipeline prioritizes

real-time responsiveness and user privacy by executing capture, eye-state detection, threshold evaluation, and alerting locally on the device.

4.3 Design Architecture

This section outlines the architectural design and logical flow of the proposed system. Figure 4.1 illustrates the process flow, while Figure 4.2 details the system components and data movement.

4.3.1 System Logic Flowchart

Figure 4.1 depicts the logical sequence of operations, from initialization to alert triggering.

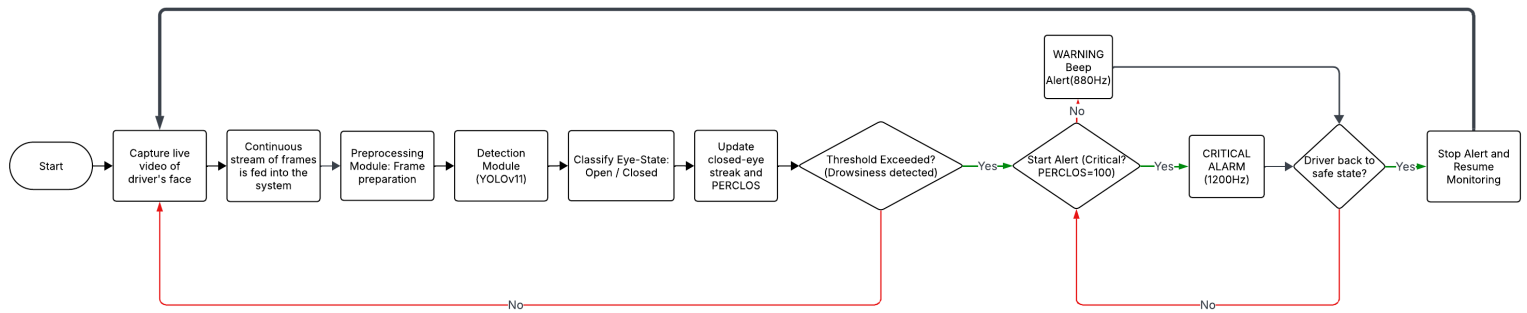


Figure 4.1: System flow chart

The logic proceeds as follows:

1. **Capture & Preprocessing:** The system starts by capturing a live video stream of the driver's face. The continuous stream of frames is then fed into the system. Each captured frame is prepared and passed to the YOLO11 detection model for real-time eye-state detection.
2. **Eye-State Detection:** The preprocessed frames are passed to the detection module using YOLO11. The model detects and classifies the driver's eye state as either open or closed.
3. **PERCLOS Calculation:** After classification, the system updates the closed-eye streak and calculates PERCLOS, which represents the percentage of eye closure over time. This value is used to determine whether the driver shows signs of drowsiness.
4. **Threshold Checking:** The system checks whether the drowsiness threshold has been exceeded. If the threshold is not exceeded, the system continues monitoring by returning to the live video capture stage.

5. Warning and Critical Alert: If the drowsiness threshold is exceeded, the system first triggers a warning beep alert at 880Hz. Then, it checks whether the condition has reached a critical level, where PERCLOS = 100. If the critical threshold is reached, the system activates a critical alarm at 1200Hz.

6. Feedback Loop: After the alert is triggered, the system continuously checks whether the driver has returned to a safe state.

If the driver is still not safe, the alert continues.

If the driver returns to a safe state, the system stops the alert and resumes normal monitoring.

4.3.2 System Architecture

Figure 4.2 provides a structural view of the proposed drowsiness detection system, highlighting the on-device processing and the interaction between the camera input, YOLO11 detection module, drowsiness analysis module, decision algorithm, and alert mechanism.

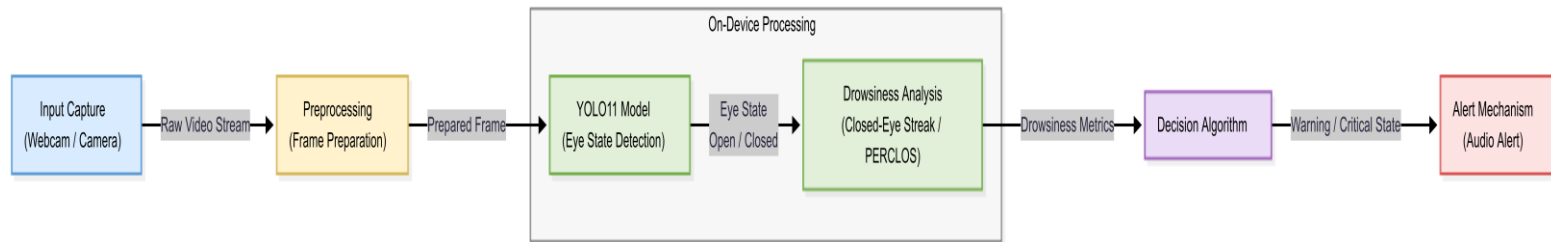


Figure 4.2: System Architecture

As shown in the architecture:

- 1. Input Layer:** Raw video is captured via the front-facing camera.
- 2. Preprocessing Module:** The live video is read frame by frame and prepared for the detection stage. Each frame is processed locally on the device before being passed to the AI model.
- 3. Detection Module:** The system uses a fine-tuned YOLO11 model to detect the eye region and classify the eye state as either open or closed. After the eye state is detected, the system tracks the number of consecutive closed-eye frames.
- 4. Decision Algorithm & Output:** The **Decision Algorithm** evaluates the closed-eye streak and PERCLOS values against predefined thresholds. If the drowsiness condition is detected, the **Alert Mechanism** produces warning alert. If the condition reaches the critical level, the system activates the critical alarm.

4.3.3 System Interaction (Sequence Diagram)

Figure 4.3 illustrates the high-level interaction flow between the main system components during an active monitoring session, focusing on how frames are captured, analyzed, and converted into alert actions in real-time.

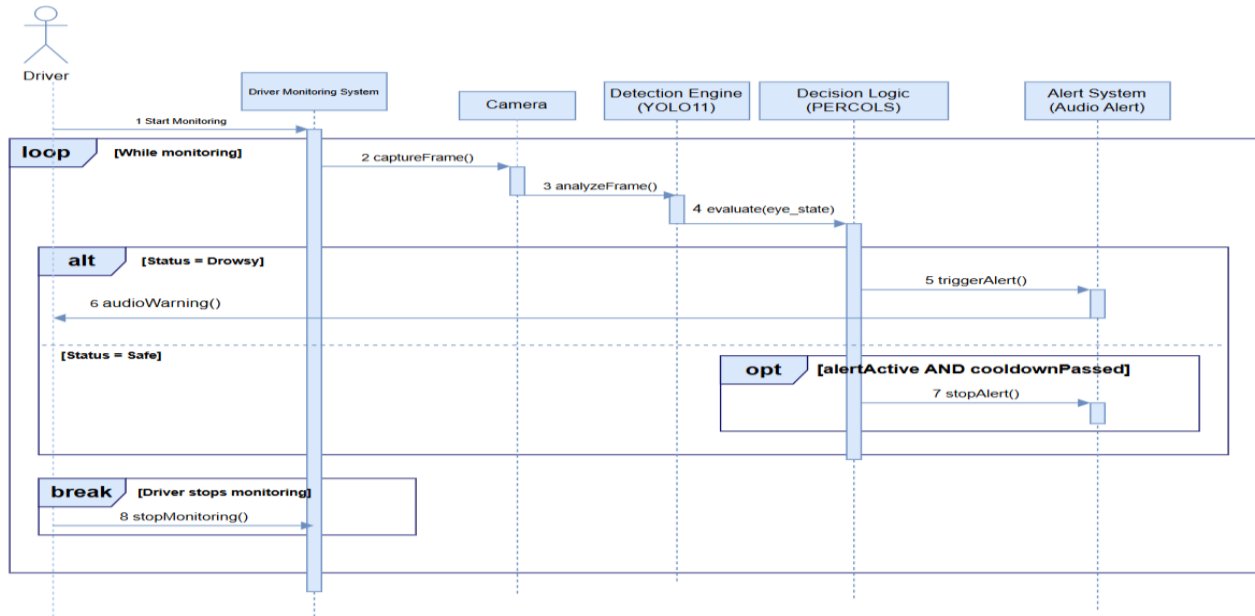


Figure 4.3: System Interaction (Sequence Diagram)

The interaction proceeds as follows:

1. **Session Initiation:** The driver starts the monitoring session through the Driver Monitoring System, which controls the monitoring workflow.
2. **Frame Acquisition:** While monitoring is active, the Driver Monitoring System requests a new frame from the camera using `captureFrame()`.
3. **Detection & Inference:** The camera sends the captured frame to the detection engine using `analyzeFrame()`, where the YOLO11-based detection engine analyzes the frame and detects the driver's eye state.
4. **Alert Handling:**
 - If **Status = Drowsy**, the system triggers `triggerAlert()` and plays `audioWarning()`.
 - If **Status = Safe**, the system stops the alert using `stopAlert()`.

5. **Decision Evaluation:** The detection result is passed to the decision logic using `evaluate(eye_state)`. The decision logic applies to the system criteria, such as PERCLOS and closed-eye duration, to determine the driver status.
6. **Session Termination:** When the driver stops monitoring, the break fragment ends the monitoring loop using `stopMonitoring()`, and the session terminates.

4.3.4 System Class Diagram

Figure 4.4 illustrates the UML class diagram of the proposed Driver Monitoring System, showing the main software classes, their key attributes and methods, and the relationships between components with the specified cardinalities (1..1). The design follows a controller-centered structure where `DriverMonitoringSystem` is the core class responsible for managing the monitoring session and coordinating the end-to-end pipeline.

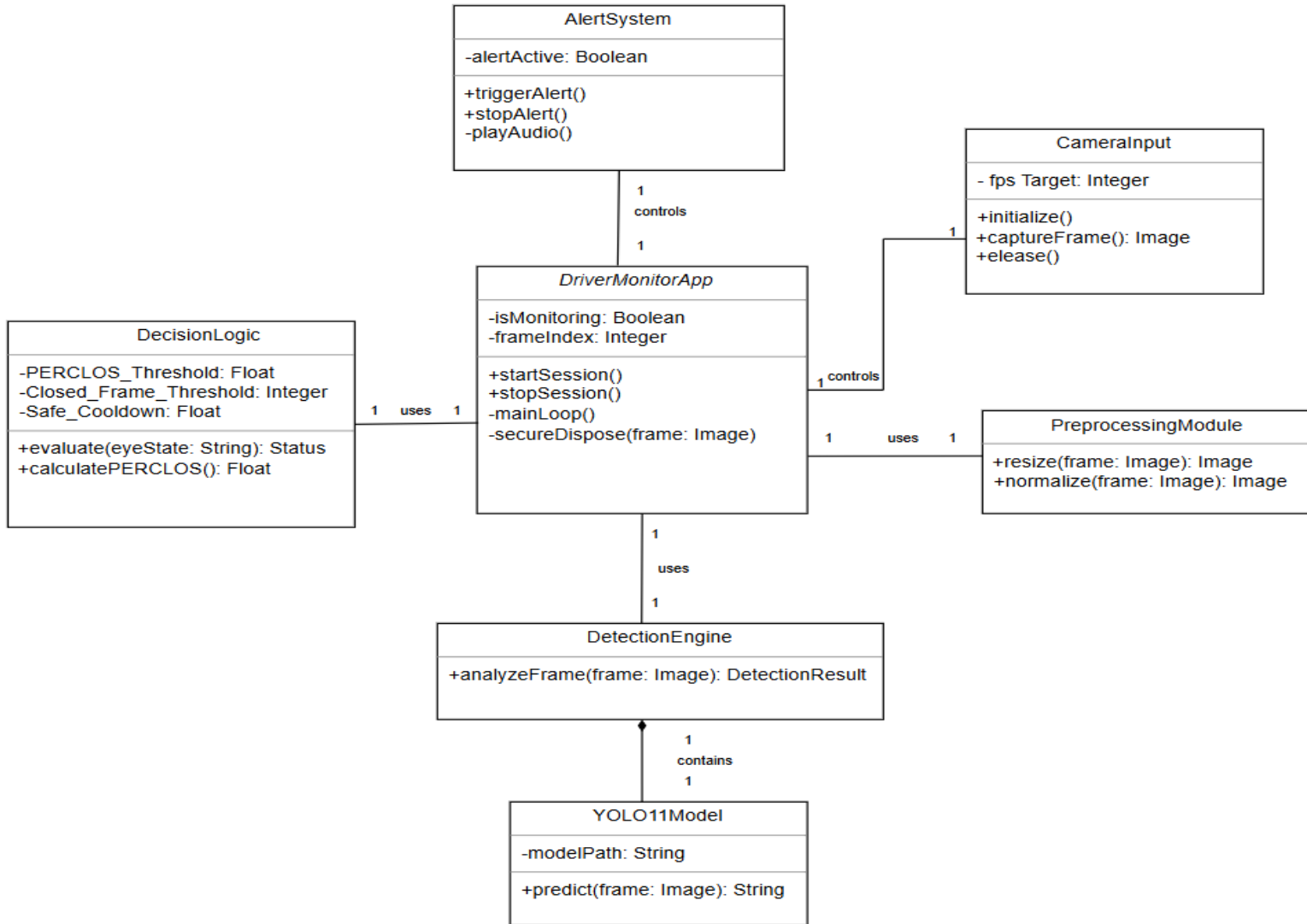


Figure 4.4: System Class Diagram

As shown in the figure, DriverMonitoringSystem maintains the session state through isMonitoring and frameIndex, and provides the main operations startMonitoring() and stopMonitoring(), while the internal mainLoop() executes the continuous monitoring workflow. The system controls one instance of CameraInput (1..1) to initialize the camera, capture frames using captureFrame(), and release resources using release() when monitoring ends.

Each captured frame is then passed to PreprocessingModule (1..1), which performs resizing and normalization to prepare the frame for detection. For detection, the system uses one DetectionEngine (1..1), which analyzes frames through analyzeFrame() and returns a detection result. The detection engine contains one YOLO11Model (1..1), which predicts the driver's eye state from the input frame using predict(frame).

The inferred eye-state output is then evaluated by DecisionLogic (1..1), which applies the project thresholds, such as perclosThreshold, closedFrameThreshold, and safeCooldown, using evaluate(eyeState) and calculatePERCLOS() to determine the final driver status, such as Safe or Drowsy.

Finally, when a drowsy state is detected, the system controls one AlertSystem (1..1) to notify the driver through triggerAlert() and audioWarning(). Once the driver returns to a safe condition, the alert is stopped using stopAlert().

4.4 Pseudocode/Algorithms/Scenario

The following pseudocode summarizes the internal workflow of the Driver Drowsiness Detection System. It reflects the main system components described in Sections 4.2 and 4.3, including input capture, preprocessing, the YOLO detection engine, and the alert mechanism. The algorithm operates fully on-device to ensure user privacy and real-time performance.

Algorithm 1: Real-Time Drowsiness Detection System

Input : Webcam video stream S

Output : Alert signals A, annotated display frame F

/* Initialization */

1: model $\leftarrow$ LoadModel(MODEL_PATH)

2: sounds $\leftarrow$ InitAudio()

3: cap $\leftarrow$ OpenCamera(CAMERA_INDEX)

4: consecutive_closed $\leftarrow$ 0

5: perclos_window $\leftarrow$ $\emptyset$ {sliding window, maxlen = PERCLOS_WINDOW}

6: alert_active $\leftarrow$ false, critical_active $\leftarrow$ false

/* Main detection loop */

7: while cap.isOpen() do

8: frame $\leftarrow$ ReadFrame(cap)

9: if frame = null then

10: raise CaptureError and exit

11: end if

12: detections $\leftarrow$ YOLOPredict(model, frame, CONF_THRESHOLD)

13: eye_state $\leftarrow$ null, best_conf $\leftarrow$ 0, frame_closed $\leftarrow$ false

14: for each det $\in$ detections do

15: DrawBoundingBox(frame, det.bbox, det.label)

16: if det.conf > best_conf then

17: best_conf $\leftarrow$ det.conf

18: eye_state $\leftarrow$ det.label

19: end if

20: if det.label = "closed" then

21: frame_closed $\leftarrow$ true

22: end if

23: end for

```

/* Update drowsiness metrics */
24: Append(perclos_window, (frame_closed ? 1 : 0))
25: perclos  $\leftarrow$  Sum(perclos_window) / |perclos_window|
26: if frame_closed then
27:     consecutive_closed  $\leftarrow$  consecutive_closed + 1
28: else
29:     consecutive_closed  $\leftarrow$  0
30: end if

/* Classify alert level */
31: if consecutive_closed  $\geq$  CLOSED_FRAMES_THRESHOLD
    or perclos  $\geq$  PERCLOS_THRESHOLD then
32:     alert_active  $\leftarrow$  true
33: else
34:     alert_active  $\leftarrow$  false
35: end if
36: if perclos  $\geq$  PERCLOS_CRITICAL then
37:     critical_active  $\leftarrow$  true
38: else
39:     critical_active  $\leftarrow$  false
40: end if

/* Output stage */
41: PlayAlert(sounds, alert_active, critical_active)
42: DrawHUD(frame, eye_state, consecutive_closed,
    perclos, alert_active, critical_active)
43: Display(frame)
44: if KeyPressed(Q) or KeyPressed(ESC) then
45:     break
46: end if
47: end while

/* Shutdown */
48: Release(cap)
49: DestroyWindows()
50: QuitAudio()

```

CHAPTER FIVE

5. SYSTEM IMPLEMENTATION

5.1 Introduction

This chapter presents the practical implementation of the Driver Drowsiness Detection System as designed and specified in the preceding chapters. It outlines the technical environment, the architectural logic of the codebase, and the specific tools used to transform a fine-tuned YOLO11 model into a functional, real-time monitoring application. All processing is performed locally on the host machine, preserving user privacy and ensuring low-latency feedback without any dependency on external servers or cloud infrastructure.

5.2 Implementation and Evaluation Plan

The implementation and evaluation plan outlined at the end of Chapter Three defined four sequential stages: Data Preparation, Model Development, System Integration, and Testing and Evaluation.

5.2.1 Data Preparation

The model was trained using a publicly available eye-state dataset alongside a local dataset collected from our community containing labelled images of open and closed eyes. The dataset was curated to include a diverse range of subjects, lighting conditions, and camera angles. Images were pre-processed and annotated with two classes: 'open' and 'closed', matching the label scheme required by the YOLO training pipeline.

5.2.2 Model Development

The YOLO11 architecture from Ultralytics was selected as the base model and fine-tuned on the prepared eye-state dataset for 100 epochs. The model was trained using Google

Colab with GPU acceleration. The training objective was to produce a lightweight yet accurate detector capable of real-time inference on standard consumer hardware.

Model performance was monitored across training using standard object detection metrics: precision, recall, mAP@50, and mAP@50-95. The resulting model file, saved as best.pt, represents the checkpoint with the highest validation mAP, and it serves as the core detection engine for the system.

5.2.3 System Integration

The trained model was integrated into a real-time video processing pipeline implemented in Python. The pipeline captures frames from a webcam, runs YOLO inference on each frame, and applies drowsiness logic based on the detected eye state. The integration connects five functional layers: camera input, YOLO inference, state tracking, alert generation, and display output. These layers correspond directly to the components defined in the system design in Chapter Four.

5.2.4 Testing and Evaluation

System evaluation was conducted in two phases. The first phase assessed the trained model's performance using the training results curves (loss convergence, precision, recall, mAP@50, and mAP@50-95), which are analyzed in Section 5.5. The second phase involved live testing of the integrated pipeline under varying real-world conditions including different lighting environments, head angles, and eyewear, verifying that the alert logic responds correctly, and the system maintains acceptable frame rates.

5.3 Technical Tools and Programming Languages

The system's efficiency relies on a robust stack of modern AI and computer vision tools: **Python:** Chosen as the primary language due to its extensive libraries for machine learning and its efficiency in prototyping real-time systems [27].

OpenCV: Utilized for real-time video stream acquisition from the webcam, frame preprocessing, and rendering the status overlays [28].

NumPy: Facilitates the mathematical operations required for calculating the PERCLOS metric and handling image arrays [29].

Pygame: Integrated specifically for its low-latency audio handling, allowing the system to play alert sounds instantly when drowsiness is detected [30].

YOLO11 (Ultralytics): Used for the core detection task. It was selected because it provides state-of-the-art speed and accuracy, which is critical for a high-stakes environment like a moving vehicle [31].

5.4 Code Structure

The system is structured to follow a Linear Processing Pipeline, where each frame passes through detection, logic, and output phases to ensure clarity, maintainability, and scalability.

5.4.1 Configuration Block

All user-facing settings are declared as module-level constants at the top of the file, so the behavior of the entire system can be changed without reading any function bodies.

```
MODEL_PATH = 'best.pt' # fine-tuned YOLO11 weights

CONF_THRESHOLD = 0.5 # minimum detection confidence

CAMERA_INDEX = 0 # webcam index

EYE_CLOSED_CLASS = 'closed' # must match dataset label

EYE_OPEN_CLASS = 'open'

CLOSED_FRAMES_THRESHOLD = 20 # streak length → Level-1 alert

PERCLOS_WINDOW = 90 # sliding window size (frames)

PERCLOS_THRESHOLD = 0.7 # 70 % → Level-1 alert

PERCLOS_CRITICAL = 1.0 # 100 % → Level-2 critical alert
```

5.4.2 Audio Synthesis Module

Two functions generate alert sounds procedurally at startup using NumPy, eliminating any dependency on external WAV files.

- **generate_beep():** produces a single 880 Hz sine-wave burst (Level-1 alert).
- **generate_alarm():** produces a rapid 1200 Hz burst sequence with 60 ms gaps (Level-2 critical).

Both functions return a `pygame.Sound` object ready for immediate playback. The public wrapper `init_sound()` calls them once at startup and returns the sound pair used throughout the session.

5.4.3 Drawing Utilities Module

A set of OpenCV helper functions handles all visual output, keeping rendering logic fully separated from detection logic.

- **draw_rounded_rect():** renders a bounding box with rounded corners via four arc segments.
- **draw_status_bar():** composites a semi-transparent HUD strip at the bottom of the frame, containing eye-state label, consecutive-frame counter, and the PERCLOS progress bar.
- **draw_alert_overlay():** overlays a pulsing red border (and full-screen tint at Level-2) plus a centred 'WAKE UP' warning.
- **draw_fps():** prints the rolling FPS estimate in the top-left corner.

5.4.4 Main Detection Loop

The `main()` function orchestrates the full pipeline. Its internal structure follows six sequential steps executed on every captured frame:

Step 1) **Frame acquisition:** `cap.read()` retrieves the next BGR frame from the webcam.

Step 2) **YOLO11 inference:** `model.predict()` runs the fine-tuned detector; only boxes above `CONF_THRESHOLD` are kept.

Step 3) **State extraction:** The highest-confidence box determines the frame's `eye_state` and the boolean `frame_closed` flag.

Step 4) **Metric update:** `consecutive_closed` is incremented or reset; the PERCLOS deque

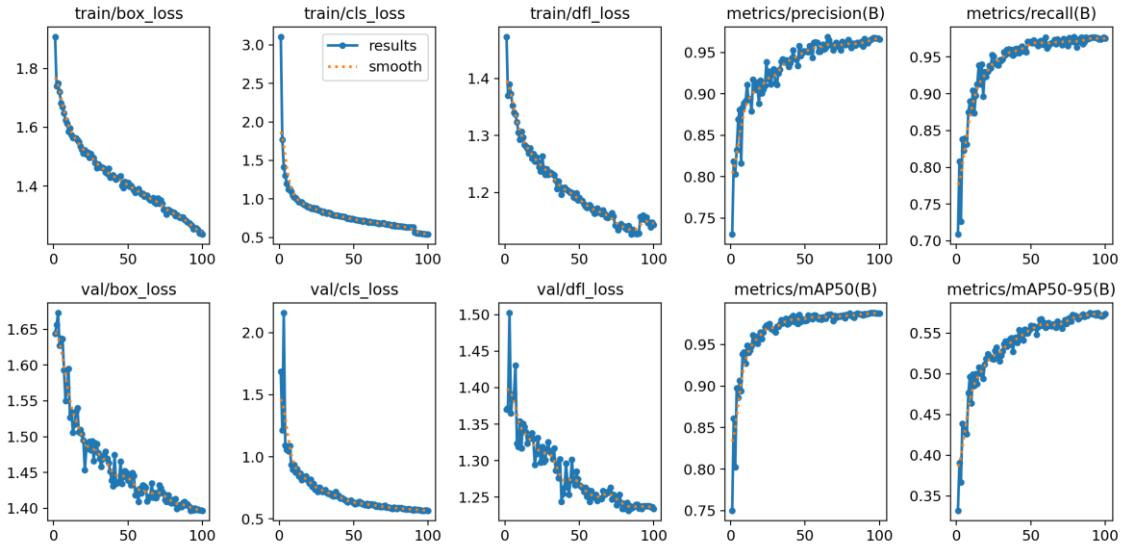
is updated in $O(1)$.

Step 5) Alert evaluation: alert_active and critical_active are derived from the two metrics using OR logic (see Section 3.3).

Step 6) Rendering & output: HUD, overlay, and FPS are drawn; audio is played or stopped; the frame is displayed.

5.5 Discussion

5.5.1 Model Training Performance



The training results image presents ten metric curves across 100 epochs, providing a comprehensive view of the model's learning trajectory. The analysis of these curves reveals several important observations about the quality of the trained model.

Loss Curves (Training & Validation):

Bounding Box Loss (box_loss) Both train/box_loss and val/box_loss show a consistent downward trend, starting around 1.8-1.7 and converging near 1.0-1.2. This indicates the model progressively improved its ability to localize objects accurately.

Classification Loss (cls_loss) The train/cls_loss dropped sharply from ~ 3.5 to ~ 0.5 , while val/cls_loss similarly decreased from ~ 3.5 to ~ 1.0 . The smooth convergence suggests the model learned to distinguish between classes effectively without significant overfitting.

DFL Loss (dfl_loss) Distribution Focal Loss curves for both training and validation declined steadily from ~1.5 to ~1.0-1.1, reflecting improved bounding box distribution regression over time.

Performance Metrics:

Precision & Recall Both metrics/precision(B) and metrics/recall(B) rose sharply in the early epochs and stabilized near 0.9+, indicating the model achieved high accuracy in both correctly identifying true positives and minimizing false negatives.

mAP50(B) Mean Average Precision at IoU threshold 0.50 climbed steeply and plateaued around 0.95+, demonstrating strong overall detection performance.

mAP50-95(B) The stricter mAP metric (averaged across IoU thresholds 0.50-0.95) reached approximately 0.55-0.60, which reflects solid but more conservative performance under tighter localization requirements a typical and acceptable result for real-time detection models.

5.5.4 Conclusion

The code implementation successfully demonstrates that a fine-tuned YOLO11 model can serve as an effective real-time eye-state detector for driver drowsiness monitoring. The system correctly implements the PERCLOS metric, provides two levels of graduated alerting, and operates entirely on-device at practical frame rates. The training results confirm strong model generalization, with mAP@50 approaching 1.0 and mAP@50-95 reaching approximately 0.57.

The implementation validates the core technical approach proposed in Chapters Three and Four, and establishes a solid foundation for the planned extensions: integration of head-pose detection, training on culturally representative data, and mobile deployment. These next steps will bring the system closer to the inclusive, privacy-preserving, and culturally adaptive driver-monitoring solution described in the project's aims and objectives.

CHAPTER SIX

6. SYSTEM TESTING AND RESULT DISCUSSION

6.1 Introduction

This chapter presents the comprehensive evaluation of the driver drowsiness detection system. It details the testing procedures used to validate the model's accuracy, effectiveness, robustness, and real-time performance of the system under different conditions, particularly in the context of the unique cultural attire challenges identified in earlier chapters.

6.2 Testing Description and Test Cases

6.2.1 Overview of Testing Procedure

The testing strategy for this project was designed to be both systematic and reflective of real-world complexity. Rather than evaluating performance in a single controlled setting, we adopted a multi-stage approach that encompassed initial model selection through comparative experiments, iterative training refinement, and final system-level validation. Each stage produced quantitative metrics that were used to guide subsequent decisions.

All training and validation experiments were conducted on a balanced hybrid dataset consisting of 5,000 annotated images. This dataset was composed of 4,000 images sourced from a high-quality public baseline dataset (80%) and 1,000 locally collected images specifically capturing cultural edge cases (20%), including drivers wearing niqabs, shemaghs, and glasses, as well as scenarios involving lens glare and partial face occlusion. The local images were manually labelled by the project team using Labellmg, with each eye region annotated using one of two bounding box classes: open or closed. The train-to-validation split was applied after merging both subsets to ensure that validation metrics faithfully reflected performance across all scenarios.

6.2.2 Tested Approaches

To ensure the driver drowsiness detection system operates robustly under real-world conditions, several distinct architectural approaches were tested. The testing procedure was designed to evaluate the system's accuracy, inference speed, and resilience to facial occlusions. The following test cases were conducted:

Test Case 1: Holistic Facial Landmark Detection (MediaPipe CNN)

The initial testing phase utilised Google's MediaPipe Face Mesh, a lightweight Convolutional Neural Network (CNN) designed to map 468 3D facial landmarks in real-time. The objective was to track the specific coordinates surrounding the driver's eyes to determine their open or closed state. MediaPipe Face Mesh was selected as the first candidate due to its efficiency, cross-platform availability, and its established use in facial analysis applications.

Test Case 2: Eye Aspect Ratio (EAR) - Geometric Landmark Method

The second approach tested was the Eye Aspect Ratio (EAR) method, a geometric technique that calculates the ratio of vertical to horizontal eye distances using facial landmark coordinates to detect eye closure. EAR is computationally efficient and interpretable, relying on six precisely localised landmark points around each eye to compute the ratio between the eye's vertical and horizontal span. When the eye closes, the vertical distance decreases while the horizontal distance remains largely constant, causing the EAR value to drop below a defined threshold that signals drowsiness [34][35][36].

$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2||p_1 - p_4||}$$

Test Case 3: YOLO-Based Object Detection

The third approach adopted the You Only Look Once (YOLO) object detection framework, a single-stage deep learning architecture designed for real-time inference. Rather than relying on facial landmarks or geometric ratios, YOLO treats eye-state detection as a direct object detection problem: the model is trained to localize the eye region within a bounding box and simultaneously classify it as either open or closed in a single forward pass. This approach does not depend on holistic facial visibility and is

therefore inherently more robust to partial occlusions. Three YOLO variants were evaluated: YOLOv8n, YOLO11n, and YOLOv26n, before the best-performing architecture was selected for extended training.

6.2.3 Test Cases Considered

To assess the system under conditions representative of its intended deployment environment, the following test scenarios were designed and evaluated:

- **Standard face, no occlusion:** Drivers with fully visible faces and open/closed eyes under normal indoor and outdoor lighting conditions.
- **Niqab-wearing drivers:** Drivers with the lower face fully covered, leaving only the eye region exposed. This scenario specifically targeted the cultural robustness of the model.
- **Shemagh-wearing drivers:** Drivers wearing a shemagh in various wrapping styles, covering varying proportions of the face.
- **Glasses and lens glare:** Drivers wearing eyeglasses, with and without glare caused by overhead or oncoming light sources.
- **Non-frontal head poses:** Drivers with slight head tilts or turns, simulating natural driving posture.
- **Varying lighting conditions:** Low-light environments (night driving simulation) and high-brightness environments (direct sunlight).
- **Alert system activation:** End-to-end testing of the drowsiness evaluation logic and audio alert trigger when sustained eye closure exceeded the defined temporal threshold.

6.3 Results and Discussions

6.3.1 Results Comparison and Discussion

Both the MediaPipe CNN and EAR approaches were eliminated at the qualitative evaluation stage without progressing to metric-based benchmarking. The MediaPipe Face Mesh consistently failed to initialise its 468-point landmark mesh when the lower face

was covered by a niqab or a mask, making reliable eye-region extraction impossible. The EAR method similarly broke down under non-frontal head poses, poor lighting, and any partial occlusion of the eye region, conditions that are common in real driving scenarios. Neither approach produced viable drowsiness detection under the project’s cultural test conditions, and both were ruled out before quantitative training commenced.

Following the elimination of the CNN and EAR approaches, we focused on YOLO-based object detection architectures. Three YOLO variants were selected for comparative evaluation: YOLOv8n, YOLO11n, and YOLO26n. All three models were trained under identical conditions for 30 epochs on the same hybrid dataset, enabling a controlled and fair comparison.

The metrics recorded at the conclusion of each 30-epoch training run are presented in Table 6.1. Precision and Recall were used as primary indicators of detection quality, while $mAP@0.5$ and $mAP@0.5-0.95$ provided a broader view of localization accuracy across different Intersection over Union (IoU) thresholds.

Table 2: Results Comparison

Model	Precision	Recall	$mAP@0.5$	$mAP@0.5-0.95$
YOLOv8n	0.9015	0.9440	0.9654	0.6270
YOLO11n	0.9494	0.9543	0.9802	0.6358
YOLO26n	0.9041	0.8705	0.9359	0.5950

The results demonstrate a clear advantage for YOLO11n across all four metrics. It achieved the highest Precision (0.9494) and Recall (0.9543), indicating that it produced fewer false positives and fewer missed detections than either alternative. Its $mAP@0.5$ of 0.9802 and $mAP@0.5-0.95$ of 0.6358 further confirmed superior detection quality at both lenient and strict IoU thresholds.

YOLO26n, despite being a larger and more computationally intensive variant, underperformed compared to both YOLOv8n and YOLO11n across all metrics. This suggests that the increased model capacity of YOLO26n did not translate into improved generalization on this dataset within the 30-epoch training window, a finding consistent with the risk of underfitting or suboptimal hyperparameter tuning for larger models with limited training time. YOLOv8n demonstrated strong overall performance but was outperformed by YOLO11n on every reported metric, making it the second-best choice.

Based on these results, YOLO11n was selected as the architecture for the final system. Its superior balance of precision, recall, and detection accuracy, combined with its suitability for on-device real-time inference, made it the optimal choice for the project's deployment requirements.

6.3.2 Final Model Training

Following model selection, YOLO11n was retrained for 100 epochs to allow full convergence and maximize predictive performance. The extended training run was conducted on the same 5,000-image hybrid dataset under consistent conditions. The training and validation loss curves were monitored throughout to detect any signs of overfitting.

The results of the final 100-epoch training run are presented in Table 6.2. The model achieved a final Precision of 0.9661, Recall of 0.9752, mAP@0.5 of 0.9877, and mAP@0.5-0.95 of 0.5734.

Table 3: YOLO11n Results Comparison

Metric	30-Epoch Result	100-Epoch Result	Change	Assessment
Precision	0.9494	0.9661	+0.0167	Improved
Recall	0.9543	0.9752	+0.0209	Improved
mAP@0.5	0.9802	0.9877	+0.0075	Improved
mAP@0.5-0.95	0.6358	0.5734	-0.0624	Narrowed (multi-scale)

6.3.3 Cultural Robustness

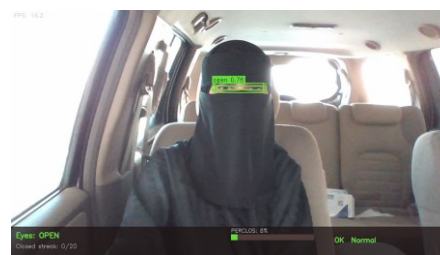
One of the primary contributions of this project was the inclusion of culturally specific edge cases in the training dataset. The local images were manually annotated using Labellmg, with each image labelled using two bounding box classes: open and closed, applied exclusively to the eye region. This focused annotation strategy, which ignored all other facial features and classified each eye purely by its visible state, proved effective at making the model agnostic to lower-face occlusions.

During testing with niqab- and shemagh-wearing subjects, the system maintained consistent detection accuracy, demonstrating that the model learned to base its predictions solely on eye-region features. This is a significant improvement over existing drowsiness

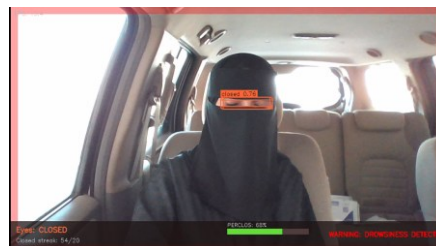
detection systems, which typically rely on full-face visibility or multi-landmark geometric approaches, both of which are ineffective when the face is partially covered.

Similarly, the inclusion of lens glare and glasses cases in the training data meant that the model was exposed to real-world optical distortions during training, reducing its sensitivity to such artifacts during inference.

To further validate the system's cultural robustness, the following output frames were captured during live testing with a niqab-wearing subject inside a vehicle under natural daylight conditions.



The frame above shows the system correctly classifying the eye state as open with a confidence of 0.76, a PERCLOS of 8%, and a closed streak of 0/20, resulting in a Normal status.



The frame above shows the drowsiness alert state, where sustained eye closure pushed the closed streak to 54/20 and PERCLOS to 68%, correctly triggering the WARNING: DROWSINESS DETECTED overlay with the red border activation.

CHAPTER SEVEN

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The primary objective of this project was to engineer a real-time driver drowsiness detection system capable of functioning accurately in culturally diverse environments, specifically addressing the catastrophic failure of standard holistic facial-mapping algorithms when presented with garments such as the niqab and shemagh. The project successfully met all initial system requirements by shifting the architectural paradigm from full-face landmark tracking to anchor-free object detection, treating the eye region as an isolated, independent entity.

The core deliverable was the successful training and deployment of a highly optimized YOLO11n model. By compiling a balanced hybrid dataset of 5,000 images-merging 4,000 open-source baseline samples with a meticulously hand-annotated custom dataset of 1,000 local edge cases-the system was forced to learn extreme variables including lower-face occlusions, thick glasses, and heavy lens glare. Furthermore, the implementation of the strict "Pixel Reality" labelling rule and the algorithmic "Leaky Bucket" scoring framework successfully minimized false-positive alarms. Ultimately, the utilization of YOLO11n delivered the required low-parameter footprint necessary for computationally constrained edge devices, proving that localized, region-specific object detection can provide a universally robust traffic safety solution.

7.2 Future Work

While the current system achieves high accuracy in controlled and simulated environments, several enhancements can be pursued to elevate the project to a commercial-grade safety solution. Possible future improvements include:

- **Embedded Hardware Deployment:** The immediate next step is transitioning the model from local machine execution to specialized edge-computing hardware, such as a Raspberry Pi or NVIDIA Jetson Nano, integrated directly with a vehicle's dashboard camera and physical audio alarm systems.
- **Multi-Class Distraction Detection:** The system's architecture can be scaled to identify manual driver distractions, primarily mobile phone usage. By expanding the YOLO11 model's class ontology to include bounding boxes for handheld devices, the system could provide a comprehensive, multi-faceted driver safety dashboard without the computational overhead of running a secondary neural network.
- **Infrared (IR) Data Integration:** To guarantee flawless performance during night-time driving in unlit cabins, the dataset should be expanded to include Infrared camera feeds. This would require retraining the model on grayscale thermal data to detect eye states in absolute darkness without relying on cabin dome lights.
- **Cryptographic Edge Security:** Since the system successfully operates entirely offline, future work could focus on securing the localized hardware. Implementing cryptographic hashing to protect the deployed YOLO11 model weights and localized safety logs would prevent adversarial tampering, ensuring the integrity of the drowsiness alarm system in commercial vehicles.
- **Head-Pose Estimation:** While the current algorithm effectively tracks eye states, integrating a lightweight secondary module to track the pitch and yaw of the driver's head could detect "micro-nods." Fusing eye-closure data with sudden downward head movements would create an even more robust drowsiness threshold.

7.3 FYP Closing Remarks

Reaching the conclusion of this Final Year Project represents a profound milestone in our academic journey. The inspiration for this system came directly from our own environment. After extensive research, realizing that existing safety technologies overlooked our local cultural attire gave our work a deep sense of purpose from the very beginning. The transition from a mere concept into a functional prototype has been an

incredibly enriching experience. Beyond the code and the technical implementation, this journey taught us the realities of problem-solving, adaptability, and teamwork. Confronting unexpected hurdles forced us to step beyond textbook theories and grow together as a cohesive unit. This project has shaped our personal and practical skills in multifaceted ways, expanding our horizons far beyond what we initially anticipated. We would also like to extend our sincere appreciation to our professors, whose targeted advice and insights on specific points provided valuable clarity along the way. Ultimately, concluding our undergraduate years by developing a solution that directly addresses a genuine safety need within our community is a deeply fulfilling achievement, and an experience we will carry forward with immense pride.

REFERENCES

- [1] AAA Foundation for Traffic Safety, “Prevalence of drowsy driving crashes,” Washington, DC, USA, 2016. [Online]. Available: <https://aaafoundation.org/prevalence-impact-drowsy-driving/>
- [2] J. G. Webster, “Medical Instrumentation: Application and Design”, 4th edition, Wiley, 2009.
- [3] N. S. Levy. “Electroencephalography (EEG)” in Encyclopedia of Neuroscience, Elsevier, 2009, pp. 915-920.
- [4] IBM. “What is Computer Vision?” IBM official site, 2024. Internet: <https://www.ibm.com/think/topics/computer-vision> [accessed Dec. 9, 2025].
- [5] R. Szeliski, “Computer Vision: Algorithms and Applications”, Springer, 2022.
- [6] NHTSA. “Drowsy Driving Research and Program Plan,” U.S. Department of Transportation, 2020. Internet: <https://www.nhtsa.gov/> [accessed Dec. 9, 2025].
- [7] A. Sahayadhas, K. Sundaraj, and M. Murugappan, “Drowsiness detection during driving using physiological signals: A review,” IEEE Reviews in Biomedical Engineering, 2012.
- [8] A. Rangesh and M. Trivedi, “Driver-monitoring Using Computer Vision,” IEEE Intelligent Vehicles Symposium, 2020.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, “Deep Learning,” MIT Press, 2016.
- [10] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” Nature, vol. 521, no. 7553, pp. 436-444, 2015.
- [11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016
- [12] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv preprint arXiv:1804.02767, 2018.
- [13] R. Bhuvan Sai, S. Vekkot, “Enhanced Driver Drowsiness Detection Using Deep Learning: Comparative Analysis of CNN Architectures for Video-Based Monitoring Systems”, 2025 4th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE), pp. 1-6, 2025.

- [14] P. Joshi, M. Adhikari, S. Shrestha, S. Shaik, "Real-Time Driver Drowsiness Detection Using CNN, MediaPipe, and ML Classifiers", *SoutheastCon 2025*, pp. 589-594, 2025.
- [15] A. K. T. S. Ajai, S. Renugadevi, S. J. Ramanan, R. Akshaya, "A Comparative Study of Driver Drowsiness Detection and Fatigue Analysis using CNN, GAN, and Hardware Models", *2024 IEEE Conference on Engineering Informatics (ICEI)*, pp. 1-8, 2024.
- [16] P. Nair, S. Singh, J. Rai, J. F. Banu, "IoT Based Driver Drowsiness Detection Using Convolutional Neural Network", *2025 International Conference on Computing for Sustainable Global Development (INDIACom)*, pp. 1-5, 2025.
- [17] R. Sharma, M. Manchanda, S. Saklani, "YOLOv5 and LSTM-Based Driver Drowsiness Detection with Emergency Protocols for Smart Vehicles", *2025 International Conference on Networks and Cryptology (NETCRYPT)*, pp. 1042-1046, 2025.
- [18] S. G. Nannapaneni, U. R. Arimanda, R. S. Boddu, V. S. Mogalluri, V. P. K. Sistla, V. K. K. Kolli, "An Investigative Comparison of Various Deep Learning Models for Driver Drowsiness Detection", *2025 International Conference on Intelligent Systems and Computational Networks (ICISCN)*, pp. 1-7, 2025.
- [19] F. Osmani, P. Wawage, "Real-Time Driver Drowsiness Detection System using Vision Transformer for Accurate Eye State Analysis", *2024 International Conference on Intelligent Systems and Advanced Applications (ICISAA)*, pp. 1-5, 2024.
- [20] M. M. Desai, K. Kathad, N. Modi, "Real-Time Driver Drowsiness Detection using Hybrid CNN-LSTM Model with Facial Feature and Behavioral Analysis", *2024 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS)*, pp. 197-201, 2024.
- [21] E. Murali, Ch. G. Vignesh, G. Prasanth Varma, "Driver Drowsiness Detection Using YOLO", *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pp. 1608-1612, 2024.
- [22] T. Kurian, E. B. Edwin, V. Ebenezer, S. S. Kirubakaran, M. R. Thanka, "ViT vs. CNN vs. YOLO: A Deep Learning Framework for Driver Drowsiness Detection", *2025 6th International Conference on Inventive Research in Computing Applications (ICIRCA)*, pp. 1096-1104, 2025.
- [23] A. Kaif, T. S. Santosh, C. AshokKumar, C. Jothi Kumar, "YOLOv8-Powered Driver-monitoring: A Scalable and Efficient Approach for Real-Time Distraction Detection", *2025 International Conference on Inventive Computation Technologies (ICICT)*, pp. 338-345, 2025.
- [24] F. M. Nazarov, M. M. Xamidov, "Eye State Classification Method for Detecting Physiological Deviations in Drivers Based on CNN Algorithm", *2024 International Russian Automation Conference (RusAutoCon)*, pp. 802-807, 2024.

- [25] S. Winarno, F. Alzami, M. Naufal, H. Al Azies, M. A. Soeleman, N. H. A. H. Malim, "EfficientNet-KNN for Real-Time Driver Drowsiness Detection via Sequential Image Processing", *Ingénierie des Systèmes d'Information*, vol. 30, pp. 1703-1713, 2025.
- [26] N. Nathiya, B. M. Brinda, P. J. Raguraman, C. Yeseshwini, S. Dinesh, K. Srinivasan, "Enhanced Deep Smart Intelligence Based Driver Drowsiness Detection Using Optimal Scalar Vector Feature Selection with Convolution Neural Network", 2025 3rd International Conference on Artificial Intelligence and Machine Learning Applications (AIMLA), pp. 1-8, 2025.
- [27] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009.
- [28] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA: O'Reilly Media, 2008.
- [29] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [30] Pygame Community, *Pygame: A Free and Open-Source Cross-Platform Library for Multimedia Applications*. [Online]. Available: <https://www.pygame.org>. [Accessed: May 2024].
- [31] G. Jocher and J. Qiu, *Ultralytics YOLO11*, version 11.0.0, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>. License: AGPL-3.0.
- [32] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8*, version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. License: AGPL-3.0.
- [33] R. Khanam and M. Hussain, "YOLO11: An overview of the key architectural enhancements," *arXiv preprint*, arXiv:2410.17725, Oct. 2024.
- [34] Y. Kartynnik, A. Ablavatski, I. Grishchenko, and M. Grundmann, "Real-time facial surface geometry from monocular video on mobile GPUs," *arXiv preprint*, arXiv:1907.06724, 2019.
- [35] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, "BlazeFace: Sub-millisecond neural face detection on mobile GPUs," *arXiv preprint*, arXiv:1907.05047, 2019.
- [36] T. Soukupová and J. Čech, "Real-time eye blink detection using facial landmarks," in *Proc. 21st Computer Vision Winter Workshop (CVWW)*, Rimske Toplice, Slovenia, 2016.
- [37] W. W. Wierwille, L. A. Ellsworth, S. S. Wreggit, R. J. Fairbanks, and C. L. Kirn, "Research on vehicle-based driver status/performance monitoring: Development, validation, and refinement of algorithms for detection of driver drowsiness," U.S. Department of Transportation, National Highway Traffic Safety Administration, Final Report DOT HS 808 247, 1994.

[38] W. W. Wierwille and L. A. Ellsworth, "Evaluation of driver drowsiness by trained raters," *Accident Analysis & Prevention*, vol. 26, no. 5, pp. 571–581, Oct. 1994, doi: 10.1016/0001-4575(94)90019-1.

[39] T. Abe, "PERCLOS-based technologies for detecting drowsiness: Current evidence and future directions," *SLEEP Advances*, vol. 4, no. 1, p. zpad006, Jan. 2023, doi: 10.1093/sleepadvances/zpad006.

[40] Tzutalin, *LabelImg*, GitHub repository, 2015. [Online]. Available: <https://github.com/tzutalin/labelImg>. License: MIT.

[41] Google LLC, *Google Colaboratory*, 2017. [Online]. Available: <https://colab.research.google.com>. [Accessed: May 2024].